Decision Analytics Journal 9 (2023) 100354

Contents lists available at ScienceDirect

Decision Analytics Journal

journal homepage: www.elsevier.com/locate/dajour

A novel Hybrid Exhaustive Search and data preparation technique with multi-objective Discrete Hopfield Neural Network

Alyaa Alway [a], Nur Ezlin Zamri [a], Mohd. Asyraf Mansor [a,*], Mohd Shareduwan Mohd Kasihmuddin [b], Siti Zulaikha Mohd Jamaludin [b], Muhammad Fadhil Marsani [b]

[a] School of Distance Education, Universiti Sains Malaysia, 11800 USM, Penang, Malaysia
[b] School of Mathematical Sciences, Universiti Sains Malaysia, 11800 USM, Penang, Malaysia

ARTICLE INFO

Keywords:
Predictive analytics
Multi-objective optimization
Discrete Hopfield Neural Network
Satisfiability logic
Data preparation

ABSTRACT

The primary objective in building predictive analytics models is to achieve optimal accuracy with real datasets. The limitations of existing models lie in their storage capacity, which hinders the progress of generating high accuracy. When a model storage capacity is limited, it may struggle to process large datasets and encounter underfitting issues, preventing it from capturing the complexities of the data. Hence, this paper addresses these challenges by introducing a novel approach to predictive analytics, focusing on expanding the storage capacity of the Discrete Hopfield Neural Network (DHNN). First, this paper employs satisfiability logic to represent the attributes of a dataset in the DHNN. This logic representation enables the model to establish a connection between neurons and attributes, enabling efficient information processing. Second, to introduce a multi-objective DHNN, a key innovation that enhances the model's storage capacity. In this context, a novel training algorithm named Hybrid Exhaustive Search is developed to optimize the DHNN's training phase. Third, this paper introduces new data preparation techniques, including feature selection and a method for identifying the best-induced logic. This best-induced logic explains and extracts dataset relationships. Finally, the proposed model is evaluated based on four reputable metrics and a variety of datasets primarily collected from the UCI Repository. The performance of the proposed model is compared with three existing models. Through extensive experiments and rigorous evaluation, the proposed model can outperform existing models in all metrics demonstrating the effectiveness of expanded storage capacity and novel data preparation techniques employed.

1. Introduction

Nowadays, most organizations demand a method that can help them grow their businesses by making strategic decisions. These decisions are based on the organization's specific goals or problems, such as exploring sales growth, reducing operational costs, and conducting market research to identify potential customers. As a result, many organizations have been seeking predictive analytic models (PAMs) to predict business trends and analyse financial or sales data [1,2]. PAMs are crucial components of data science as they assist in making predictions based on data sets. Moreover, PAMs are not limited to predicting data sets; they can also be used for clustering and classifying data sets. For example, creating an effective PAM requires integrating a machine learning model with data mining. Neural networks are effective models for PAMs due to their ability to accurately represent attributes and factors in data sets through input neurons. Additionally, artificial neural networks (ANNs) are the closest models to simulating decision-making processes since they mimic the human brain.

Today, everyone is in the era of building intelligent machines, and there is no better source of inspiration than the human brain. Humans are particularly blessed by evolution with a brain capable of performing the most complex tasks. Since the early 1940s, scientists have been trying to build mathematical models and algorithms that mimic the computations performed inside the brain. ANNs are currently among the most influential algorithms in machine learning and artificial intelligence. This is because ANNs consist of input and output layers and can also include a hidden layer (in many cases) that contains units translating the input into something usable by the output layer [3]. The latest application of ANN is in autonomous vehicles which self-driving cars depend heavily on neural networks for decision-making processes.

* Corresponding author.
E-mail addresses: alyaalway@student.usm.my (A. Alway), ezlinzamri@student.usm.my (N.E. Zamri), asyrafman@usm.my (M.A. Mansor), shareduwan@usm.my (M.S.M. Kasihmuddin), szulaikha.szmj@usm.my (S.Z.M. Jamaludin), fadhilmarsani@usm.my (M.F. Marsani).

https://doi.org/10.1016/j.dajour.2023.100354
Received 19 July 2023; Received in revised form 21 October 2023; Accepted 30 October 2023
Available online 2 November 2023
2772-6622/© 2023 The Author(s). Published by Elsevier Inc. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

From Ref. [4], this paper discussed the implementation of various ANNs to cater problems like motion control, path planning, and behaviour arbitration in self-driving cars technology. This paper also discusses the advantages and disadvantages of various ANNs for autonomous driving application. Another recent application of ANN is healthcare diagnosis and treatment. ANN has been used for conducting medical image screening analysis to guide medical specialists in diagnosing any diseases. For example, Ref. [5] provides an overview of different techniques used to detect and classify diseases from medical images like X-ray and MRI scans. Besides, Recurrent Neural Network (RNN) is a part of ANN which is specifically good at capturing temporal dependencies and patterns in sequential data [4]. The structure of an RNN is different from others. In an RNN, the data can be fed back into the input before being fed forward again for further processing and final output. In other words, RNN has the ability to gather the information from previous time steps to generate sequences at each time step. The common applications involving RNN are speech recognition [6] and time series analysis [7] due to its capability to model and generate sequences which suitable for tasks like text translation. This is different from a feedforward neural network, which passes the data forward from input to output. Discrete Hopfield Neural Network (DHNN) is one type of RNN that capable at store and retrieve multiple patterns [8]. Considering this as its capability, many researchers implemented DHNN in several applications like pattern recognition [9] and optimization problems [10]. In addition, the characteristics of DHNN with respect to its structure are good memory storage called Content Addressable Memory (CAM), symmetric synaptic weight, and energy minimization. Generally, CAM is a computer memory system which able to recall entire memories in a system. Meanwhile, DHNN specifically implement CAM for storing binary input patterns and connections which makes DHNN valuable model for pattern retrieval tasks [11]. Even with the presence of noise pattern in a data, CAM is able to recall entire memories. The goal of DHNN is minimizing the energy function to ensure the correctness of the final neuron states. During the testing phase of DHNN, the network will keep iteratively updates the neurons states until the final neuron states achieved stable state. The correct final neuron states certify that the energy function of DHNN always converges towards the global minima energy [12]. Additionally, Ref. [13] stated that the characteristics of synaptic weight in DHNN that are no self-connection and symmetrical will always ensure the stability of DHNN. The paper also added that the neurons must be updated in an asynchronous manner to improve the stability and convergence of DHNN. To achieve robust stability in DHNNs, it is imperative to discover a symbolic representation that effectively captures the behaviour of the binary neurons within the DHNN model. As a result, this paper will employ satisfiability logic as a symbolic representation for the neurons in the DHNN model.

Boolean satisfiability or referred as SAT, is a mathematical logic used mainly used in computer science. It consists of two important logical formulas which are variables (either TRUE of FALSE) and operators (AND, OR, and NEGATION). The structure of SAT is expressed in Conjunctive Normal Form (CNF) [14]. SAT is used to determine whether a satisfying assignment to the variables exists, making the entire logic satisfiable. Apart from its theoretical significance, SAT finds applications in various fields such as artificial intelligence, data mining, and optimization problems [15]. In the context of solving optimization problems, SAT can be employed to solve complex problems. Generally, both SAT and DHNN share the same relevance in solving optimization tasks. However, DHNN is a non-symbolic form of a model that does not rely on any symbols or operators. Information in DHNN is distributed through interconnected neurons only. Therefore, this situation explains that SAT representation is needed in DHNN to encode the interconnected neurons with a SAT logical structure. In this representation, each neuron in the DHNN model represents a variable in the SAT logic. Additionally, the incorporation of SAT in DHNN capable in resolving complex solution space, enhancing its functionality as an effective model for tackling optimization challenges with enhanced computational efficiency. There are several developments have emerged involving SAT representation in the DHNN model. The earliest work by Ref. [16] founded the utilization of Horn-SAT to symbolize the DHNN model. This work involved a calculation of synaptic weight by comparing the cost function and the Lyapunov energy function. This calculation determines whether the network had reached the global minimum energy state. Building on this foundation, Ref. [17] introduced systematic k Satisfiability (kSAT) where $\mathtt{k} = 2; 3$ similarly to 2 Satisfiability (2SAT) and 3 Satisfiability (3SAT). While 2SAT and 3SAT restrict the number of literals per clause, this work paved the way for more flexible representations. Genetic Algorithm (GA) and Exhaustive Search algorithm (ES) were employed as training algorithms in DHNN to discover satisfying interpretations of kSAT. Notably, the results demonstrated that the DHNN model with GA outperformed the benchmark ES, due to the presence of effective search operators. The success of these studies encouraged further exploration. Ref. [18] proposed a non-systematic variant known as Major 2 Satisfiability (MAJ2SAT) that specifically formulated to introduce the ratio of major clause. MAJ2SAT combines both 2SAT and 3SAT structures with condition 2SAT clauses should be more than 2SAT clauses in a structure. This innovation allowed for the minimization of cost functions with a higher number of learning iterations in DHNN and increase the variation of final neuron states. In another direction, Ref. [19] introduced non-systematic Weighted Random 2 Satisfiability (r2SAT) and incorporated first and second-order logic into the SAT representation. By utilizing Genetic Algorithm (GA) in the logic phase, this work effectively formulated r2SAT, leading to superior performance compared to both systematic SAT and other non-systematic SAT representations in terms of producing global solutions. The training phase of DHNN model when incorporated with SAT logic (DHNN–SAT model) can be optimized with the inclusion of training algorithms. However, previous experiments utilizing single objective DHNN without training algorithm have revealed significant shortcomings, particularly in achieving minimization of cost function of the DHNN–SAT model [20,21]. This is due to the reason no training algorithm implemented to optimize the training phase. besides, the utilization of single objective has restricted the model to fully explore the entire search space. These limitations highlight the need for a more effective approach in advancing the DHNN–SAT model's evolution. To bridge this gap and enhance its performance, the incorporation of multi-objective functions with effective training algorithm will not only address the identified limitations but also provide a new way to explore a broader solution space which help to create more robust PAM.

Importantly, it should be noted that the elements in the data preparation phase of a PAM play a crucial role in achieving high accuracy. In the context of the DHNN–SAT model, several developments have been made to enhance its capability for predictive analytics with real datasets. These progresses encompass modifications to the data preparation phase, including attribute selection methods, cross-validation techniques, and the train–test split ratio. The earliest work to integrate the DHNN–SAT model for predictive analytics is referenced in Ref. [22], where a logic mining model called 2 Satisfiability based reverse analysis (2SATRA) was introduced. The proposed 2SATRA utilized 2SAT to extract single induced logic that able to describe the decision of a dataset. In this work, it was found that the 2SATRA model yielded high-quality induced logic, enabling accurate predictions of League of Legends game outcomes for three different regions. Ref. [23] extended 2SATRA by permutating various attribute arrangements when analyzing commodity prices. This strategic approach allowed 2SATRA to increase the probability of obtaining higher-quality induced logic, ultimately leading to optimal results. As a result of these two studies, the 2SATRA model was able to achieve an accuracy rate of 70% or higher, even without the implementation of cross-validation and proper attribute selection methods. Motivated by the promising results achieved by Ref. [23], Ref. [24] took the initiative to introduce a

permutation operator into 2SATRA, which is referred to as P2SATRA. The permutation operator is added to explore a wider solution space, depending on the number of attributes selected for the study. A higher number of attributes chosen leads to an increased likelihood of achieving optimal accuracy. P2SATRA was able to outperform 2SATRA and other two models based on accuracy for all 11 real datasets. It is worth mentioning that P2SATRA shares the same similarities with 2SATRA, as they both implement random attribute selection and a 60:40 train–test split ratio. Recognizing the opportunity to further enhance the logic mining model, Ref. [25] introduced the first attribute selection method in the DHNN–SAT model, named the Supervised Logic Mining model (S2SATRA). S2SATRA utilizes correlation analysis to select significant attributes and maintains the 60:40 train–test split ratio. The proposed S2SATRA was tested with various performance metrics in real-life datasets. Based on the simulations, S2SATRA outperformed 2SATRA in handling real-life datasets for all accuracy, precision, sensitivity, F1-score, and MCC. This finding showcased the method in choosing attributes effectively managed to improve the performance of logic mining model. This perspective inspired Ref. [26] to introduce A2SATRA, where log–linear analysis was employed as the attribute selection method in the data preparation phase to extract the final induced logic. The results reveal that A2SATRA achieved highest average accuracy of 90.4% compared to 2SATRA, P2SATRA, and S2SATRA. Through the examination of these models, it becomes evident how logic mining approaches have continuously improved, primarily driven by advancements in the data preparation phase. Notably, the evolution of attribute selection methods, from random selection in 2SATRA and P2SATRA to the utilization of correlation and log-linear analysis, underscores the progress in enhancing these models. This finding demonstrates that an effective data preparation phase can enhance the performance of the PAM model. Besides, while the previously mentioned logic mining models have demonstrated good performance in terms of accuracy and successfully generated optimal induced logic, they still encounter overfitting issues and become trapped in local minima solutions when dealing with large datasets. As mentioned earlier, conducting cross-validation helps the model explore a broader search space. Therefore, there is a need for a new technique capable of exploring a wider search space while employing effective data preparation methods. To address all these challenges, this paper proposes a novel predictive analytics model based on the best attribute selection method, cross-validation, optimal selection of induced logic, a robust training algorithm, and multiple storage capacities. Thus, the contributions of this paper are outlined as follows:

- To propose a predictive analytic model with the mechanism of Reverse Analysis method in Discrete Hopfield Neural Network. The proposed model utilized Major 2 Satisfiability as the logical rule to govern network. The computation of power best logic is based on highest summation of true positives and true negatives of the logical rule to the class of the training dataset.
- To implement multi-objective function in the training phase of Discrete Hopfield Neural Network. The training phase of the model will train the power best logic with respect to both fitness and diversity which corresponds to multiple units of Content Addressable Memory of the network.
- To propose a new training algorithm namely Hybrid Exhaustive Search to optimize the training phase of the Discrete Hopfield Neural Network. The proposed algorithm consists of a hybrid optimization operator of intelligent mutation which was inspired by the conventional Genetic Algorithm. The intelligent mutation operates by diversifying neuron states in the Discrete Hopfield Neural Network to ensure optimal synaptic weight management.
- To compare the performance of the proposed predictive logic mining model with other baselines based on diverse datasets from various fields. Four crucial evaluation metrics were employed in the experimentation which are accuracy, precision, sensitivity, and the Matthews Correlation Coefficient.

2. Major 2 Satisfiability in Discrete Hopfield Neural Network

In this paper, we employ the term SAT logic which is defined as the utilization of Boolean SAT as a logical structure to model the behaviour and connections of neurons in the network. The use of SAT logic in this study is inspired by its capability to represent complex logical relationships among neurons. By applying the principles of SAT in DHNN, we can gain more insights into the network's behaviour through synaptic weight patterns. In the context of SAT logic in DHNN, there are two types of SAT which are systematic and non-systematic SAT logical structure. Systematic SAT refers to a structured logical structure that consists of only one type of SAT for example 2SAT and 3SAT. Meanwhile, non-systematic SAT refers to a logical structure with more than one type of SAT. The differentiation of these two types of SAT helps us to evaluate the model, especially in terms of the exploration of search space capacity in dealing with large data. Among non-systematic SAT representations, Major 2 Satisfiability (MAJ2SAT) stands out as an optimal choice for enhancing DHNN performance due to its inclusion of high-order clause interactions [27]. There are several convincing reasons for adopting high-order logic in DHNN. Firstly, traditional DHNN face limitations in terms of storage capacity due to their reliance on a single CAM. It is important to note that, the synaptic weight values in DHNN are stored in the CAM in the matrix form, not in terms of individual neurons or connections. However, by implementing higher-order logic within DHNN, memory capacity can be significantly expanded, allowing for more precise programming of memories and patterns. This increase in capacity opens opportunities for DHNN to tackle more complex tasks. Secondly, high-order logic plays a vital role in improving the generalization capabilities of DHNN. By capturing more complex relationships between variables, DHNN trained with high-order logic exhibits the potential to generalize better to unseen data, making them more adaptable and versatile for various applications. MAJ2SAT logic or referred as $'_{MAJ2SAT}$ comprises of second order, $M_i^{(2)}$ and third order, $N_i^{(3)}$ clauses [18]. Unlike systematic SAT logic (2SAT & 3SAT), non-systematic $'_{MAJ2SAT}$ focuses on the exploration of potential solutions in two different types of clauses with higher search space. $M_i^{(2)}$ and $M_i^{(3)}$ comprise of two and three literals in each clause respectively. The value for literals can be either $a_i = 1$ (True) or $a_i = -1$ (False). This logic offers a flexible and adaptable technique in tackling complex problems. Eq. (1) shows a general formula for $'_{MAJ2SAT}$ which must satisfy the condition set in Eq. (2). Eq. (2) explains that the number of 2SAT clauses (p) must be more than the number of 3SAT clauses (q) in $'_{MAJ2SAT}$.

$$'_{MAJ2SAT} = \bigwedge_{i=1}^{p} M_i^{(2)} \bigwedge_{i=1}^{q} M_i^{(3)} \tag{1}$$

$$= \frac{p}{p+q} \tag{2}$$

It is important to note that, the value in $'_{MAJ2SAT}$ must be within 0.5 and 1. If $= 0{:}5$, the logic representation is similar to RAN3SAT logic. Meanwhile, if $= 1$, the logical representation only consists of second order clauses which similar to 2SAT logic. In this paper, $'_{MAJ2SAT}$ plays an important role in representing the attributes of a dataset. Therefore, a model is needed to carry out the information of attributes in the dataset to finally interpret the relationship between attributes. DHNN is a specific type of ANN that incorporates a feedback mechanism, where the network's output is transmitted back into the network itself. DHNN is unique which does not have any hidden layers [19] and is particularly well suited for solving various optimization problems. In DHNN, each neuron in the network is represented as a bipolar neuron, with its state denoted by either 1 or –1, corresponding to the interpretation of true and false, respectively. In this paper, the literals in MAJ2SAT will represent the neurons in DHNN. The update process for the neurons, S_i in DHNN follows Eq. (3) where $W_{[i;j]}$ is the synaptic weight from unit i to j.

$$S_i = \begin{cases} 1; & \text{if } \sum_j W_{[i;j]} S_j \geq 0 \\ -1; & \text{otherwise} \end{cases} \tag{3}$$

Generally, DHNN is divided into two phases, the training phase and testing phase. In the training phase, Eqs. (4)–(5) plays a crucial role in evaluating the cost function $'_{MAJ2SAT}$ [18]. The objective of minimizing the cost function is to ensure that DHNN model retrieve the optimal synaptic weight. There are two important characteristics of synaptic weights which are symmetrical weight $W_{[1;2]} = W_{[2;1]}$ and no self-connection $W_{[1;1]} = W_{[2;2]} = 0$ as stated in [12].

$$E_{'_{MAJ2SAT}} = \frac{1}{4}\sum_{j=1}^{H}\left(\sum_{i=1}^{I} Q_i\right)_j + \frac{1}{8}\sum_{j=1}^{H}\left(\sum_{i=1}^{I} Q_i\right)_j \tag{4}$$

$$Q_i = \begin{cases} 1 + S_{a_1}; & \text{when } a_1 \\ 1 - S_{a_1}; & \text{when } \neg a_1 \end{cases} \tag{5}$$

In this work, maximum number of satisfied clauses in $'_{MAJ2SAT}$ corresponds to the successful minimization of $E_{'_{MAJ2SAT}}$. Therefore, optimal training phase leads to $E_{'_{MAJ2SAT}} = 0$. Due to the properties of $'_{MAJ2SAT}$ being an NP-problem, there are multiple possibilities of interpretations that would make $'_{MAJ2SAT}$ satisfied. In order to select which satisfied interpretations qualify as the optimal neuron connections, the proposed DHNN will optimize another objective function in the training phase with respect to the diversity of $'_{MAJ2SAT}$ satisfied interpretations. An existing study by Karim et al. (2022) has showed a successful experimentation in considering these two objectives (fitness & diversity) in DHNN. Karim et al. (2022) reported that both objective functions are crucial to allow DHNN in memorizing other optimal neuron connectivity which lead to the search of optimal synaptic weights in magnitude and negativity. In comparison to the traditional DHNN, the proposed DHNN possessed higher storage capacity of CAM. In this context, if *N* strings of $'_{MAJ2SAT}$ satisfied interpretations were found, the maximum number of CAM in the proposed DHNN is *N*. In the testing phase, the evaluation of the final neuron states involves two steps. Firstly, the local field equation is computed, which can be found in Eq. (6) below. Secondly, the neuron states are filtered through the hyperbolic tangent activation function (HTAF) as shown in Eq. (7). Notably, HTAF is chosen due to its capability to reduce neuron oscillations which will produce optimal final neuron states. The process continues until the proposed model reaches the global minimum energy, as indicated by Eq. (8). This signifies that the DHNN converged to a stable state and the final neuron states have been determined. Noted that $H_{'^{min}_{MAJ2SAT}}$ is the minimum energy of the network and $H_{'_{MAJ2SAT}}$ is the Lyapunov energy function [27]. This understanding of DHNN behaviour forms the foundation for the subsequent exploration of the training algorithm's role in ensuring to attain global minimum energy.

$$h_i(t) = \sum_{k=1;j\neq k}^{}\sum_{j=1;j\neq k}^{} W^{(3)}_{[i;j;k]} S_j S_k - \sum_{j=1;j\neq i}^{} W^{(2)}_{[i;j]} S_j + W^{(1)}_{[i]} \tag{6}$$

$$S_i(t) = \begin{cases} 1; & \tanh\ h_i \geq 0.001 \\ -1; & \tanh\ h_i < 0.001 \end{cases} \tag{7}$$

$$\left(H_{'^{min}_{MAJ2SAT}} - H_{'_{MAJ2SAT}}\right) \leq f \tag{8}$$

3. Hybrid Exhaustive Search algorithm

The combination of DHNN–SAT and the advanced training algorithm empowers the network to learn and adapt, effectively capturing complex patterns and optimizing its synaptic weights. Therefore, an effective training algorithm is essential to enhance the performance of the proposed model and its potential to solve a wide range of problems. This section will thoroughly discuss a novel training algorithm named Hybrid Exhaustive Search (HES), designed to optimize the training phase of DHNN by fulfilling all multi-objective functions. Previously, the DHNN model focused solely on solving a single objective function, which was to minimize the cost function. However, issues persisted due to the limited search space and the lack of diversified neuron states, resulting in suboptimal results. To address these challenges, this section will present the definition of multi-objective functions and introduce the structure of the HES algorithm.

3.1. Multi-objective functions

In this subsection, we will introduce the concept of multi-objective functions, which play a vital role in enhancing the DHNN's model capabilities. In the modified multi-objective DHNN, the incorporation of multiple CAMs fundamentally modifies the memory storage and retrieval processes. Unlike the traditional DHNN, which contains one CAM, the modified DHNN utilizes multiple CAMs to handle different logical outputs simultaneously. When the DHNN processes input data, it produces logical outputs, each representing a distinct solution. These outputs are called 'power strings', and they are stored in separate CAMs. The power strings play a vital role in the modified multi-objective DHNN, where they must satisfy all the objective functions, including the maximum fitness function and diversity function. If a string can only satisfy one of the objective functions or neither, it is not considered a power string. Mathematically, the proposed multi-objective function is formulated as shown in Eq. (9).

$$\text{Maximize}: f(F, D, \mathcal{H}) \tag{9}$$

The definition and general formula of each objective is explained as follows.

(a) Fitness function (F): To compute satisfied interpretation of $'_{MAJ2SAT}$ logic. This is to ensure that the logic is able to achieve maximum fitness throughout the simulation. The optimal value for fitness function is the total clauses in $'_{MAJ2SAT}$ which equal to p+q. Eq. (10) represents the formulation of maximizing fitness function in Eq. (11). Meanwhile, Eq. (12) explains the value for satisfied and unsatisfied clauses.

$$\max[F] \tag{10}$$

$$F = \sum_{i=1}^{} M_i^{(2)} + \sum_{i=1}^{} M_i^{(3)} \tag{11}$$

$$M_i^{(k)} = \begin{cases} 1; & \text{satisfied} \\ 0; & \text{unsatisfied} \end{cases} \quad \text{where } k = 2;3 \tag{12}$$

(b) Diversity function (D): To calculate the number of diversified neuron states in $'_{MAJ2SAT}$ logic. The objective of proposing this objective is to create more diversification and differentiation in the model which can contribute to improved robustness and fault tolerance. The optimal value for diversity function is any value that is more than 10% of total literals in $'_{MAJ2SAT}$ which equal to $0{:}1(2p+3q)$. Eq. (13) represents the formulation of maximizing diversity function in Eq. (14). Meanwhile, Eq. (15) explains the diversity value is calculated as 1 when there is a difference between initial neuron state, a_1^* and generated neuron state a_1.

$$\max[D] \tag{13}$$

$$D = \sum_{i=1}^{2p+3q} d_i \tag{14}$$

$$d_i = \begin{cases} 1; & a_1^* \neq a_1 \\ 0; & a_1^* = a_1 \end{cases} \tag{15}$$

(c) Power strings production $\mathcal{H}$: To produce top power strings that both achieve maximum fitness function and diversity function. This objective is initiated to allow the proposed model train larger and complex data. Having more power strings enables the model to have more CAM which allows for efficient storage and retrieval of optimal results. The optimal value for power strings is five strings. Eqs. (16)–(17) represents the formulation of maximizing power strings where it must satisfy both maximum fitness and maximum diversity.

$$\max\ \mathcal{H} \tag{16}$$

$$\mathcal{H} \in (\max[F]; \max[D]) \tag{17}$$

In the context of Pareto front which commonly associated with multi-objective functions, this work disregards any trade-offs between the two objective functions formulated in Eq. (17). The reason why this approach was considered is because both objective functions all equally important to ensure optimal production of power strings. If the fitness of power strings is waived, suboptimal synaptic weights produced. Consequently, the proposed DHNN will trap in local minima. On the other hand, the defined diversity tolerance for Eq. (13) is only 10%. Any lower diversity tolerance will diminish any possibility of multiple units CAM that expand the search space of the proposed DHNN. A new specialized algorithm namely HES is proposed whereby all optimization components in the algorithm are organized to effectively optimize Eqs. (16)–(17). Based on the above explanation, the DHNN model has been modified with more than one CAM in the system. Through production of power strings, multiple CAM are generated with its respective space to retrieve final neuron states. For example, in this paper, the maximum number of power string can be generated by the model are five. The significance of these power strings lies in their representation of diverse and optimal solutions to the given problem. In our model, each power string is stored in its respective CAM, allowing the DHNN to efficiently handle multiple objectives and store different optimal solutions simultaneously. This feature enhances the model's adaptability and versatility in handling complex tasks by providing a range of solutions that can be accessed based on specific requirements. Worth mentioning that the proposed model will have maximum of five CAM. Each CAM will store its respective power string which has its own testing phase. In most cases, the DHNN model often encounters local optima, where the network becomes trapped in suboptimal states when dealing with large datasets. Therefore, the use of HES algorithm is necessary to optimize the performance of the training phase in DHNN and ensure all objective functions are satisfied.

3.2. Stages in Hybrid Exhaustive Search algorithm

As mentioned previously, the HES algorithm is required to optimize multi-objective functions (fitness & diversity) formulated in Eqs. (16)–(17). Both objective functions are optimized to ensure optimal training phase of DHNN. HES is inspired by the combination of two distinct mechanisms from different algorithms. It inherits the 'trial and error' searching strategy from the ES algorithm and the local search operator mutation from the GA algorithm. One may ask why we do not use existing ES or GA to optimize the training phase. First, although trial and error searching allow for a broad exploration of the search space, ES suffers from high computational costs [28]. It often requires a large number of evaluations and takes a longer time to find a suitable solution. Therefore, ES is impractical for solving multi-objective functions. Second, GA has a powerful mutation operator that can disrupt the current population and escape local optima solutions. However, the mutation operator in Ref. [17] randomly chooses a solution to mutate instead of improving the selected unsatisfactory solution. Therefore, HES has developed a new searching operator named 'intelligent mutation' in the training phase of DHNN. Intelligent mutation helps to facilitate the exploration of the solution space thus helps finding optimal solutions in a timely manner. The implementation of HES in DHNN model with $'_{MAJ2SAT}$ is presented as follows:

Stage 1: Population Initialization

Generate the initial population (P) by randomly assigning solutions. Let $P = S_1; S_2; S_3; \ldots; S_i$ where S_i is ith solution. Then, evaluate the fitness of each solution.

Stage 2: Intelligent Mutation

Here, the solutions with $F < p + q$ will undergoes intelligent mutation phase. This operator will identify the unsatisfied clause which caused to $'_{MAJ2SAT} = -1$. The detection process for $M_i^{(k)} = -1$ is an integral part of exploring and maximizing the fitness function. Then, the operator will mutate the neuron states based on $M_i^{(k)} = -1$ only. This process is to ensure that all solutions were mutated towards maximum fitness. Furthermore, the iteration will stop until it reaches maximum number of generations.

Stage 3: Fitness Evaluation

The solutions will be evaluated based on Eqs. (9)–(11). The iteration will stop until all solutions achieved maximum fitness, $\max[F] = p + q$.

Stage 4: Diversity Assessment

The diversity neuron states will be computed whether it achieved maximum diversity or not by referring to Eqs. (12)–(14). All set of solutions should achieve more than $0{:}1\,(2p + 3q)$.

Stage 5: Production of Power Strings

In this significant process, the proposed model will only select the top five solutions with $\max[F; D]$. The selection process for the top power strings is based on their uniqueness from each other and their higher proportion of positive neuron states.

The system will halt when it reaches the maximum number of training iterations and identifies the top power strings. If the proposed model fails to find the top power strings, the process will be restarted by initializing a new population. By combining the DHNN–SAT model with the highly efficient HES training algorithm, we unlock the potential to create a PAM model. This combination leverages the network's robust CAM capabilities, enabling it to excel in advanced data analysis, prediction, and modelling tasks with exceptional accuracy and efficiency.

4. The proposed predictive analytic model

PAM is known as a model designed to predict or classify real datasets. However, the effectiveness of PAM depends on the characteristics of the data preparation phase, such as the ratio used to split the training and testing data, as well as the attribute selection method. In this section, we will discuss a PAM model named 2HESRA, which utilizes $'_{MAJ2SAT}$ in the DHNN with the HES algorithm by implementing reverse analysis technique. Reverse analysis is a traditional technique used to describe the decision process of a real dataset. This technique involves the generation of the best logic (training phase) and the final induced logic (testing phase). The production of the best logic $'_{best}$ is crucial as it demonstrates the optimal connections between neurons with the correct synaptic weight values. Meanwhile, the final induced logic $'^{final}_{induced}$ represents the decision made for a real dataset. For the sake of clarity, we will focus on explaining selected differences between the 2HESRA and 2SATRA models, rather than providing an overall comparison for all models. Specifically, there are four differences between 2HESRA and 2SATRA. First, 2HESRA capitalizes on correlation analysis [25] for attribute selection, while 2SATRA chooses attributes randomly. Second, 2HESRA uses 5-fold cross-validation [29], whereas 2SATRA does not perform any cross-validation on the dataset. Third, in 2SATRA, the best logic $'_{best}$ is generated based on the highest frequency of second-order clauses for the training datasets, equal to 1 $'_{train} = 1$. On the other hand, 2HESRA calculates the value of True Positive (*TP*) and True Negative (*TN*) for the overall training datasets, considering both 1 and –1 $'_{train} = 1; -1$. Finally, 2SATRA selects the optimal induced logic $'^{final}_{induced}$ that explains the relationship between goal and attributes of the dataset based on the highest number of successes $'_{induced} = '_{test}$ [16]. Meanwhile, 2HESRA chooses $'^{final}_{induced}$ based on highest accuracy. It is worth mentioning that there are many differences between the proposed 2HESRA and existing logic mining models other than 2SATRA. Table 1 summarizes the characteristics 2HESRA and other models. Next section explains step-by-step process for the 2HESRA model.

Table 1
The differences between 2HESRA and existing logic mining models.

Model	Attribute selection method	Cross validation	CAM	Best logic method	Induced logic method	Permutation operator
2SATRA [22,23]	Random	No	Single	Frequency	Accuracy	No
P2SATRA [24]	Random	No	Single	Frequency	Accuracy	Yes
S2SATRA [25]	Correlation analysis	No	Single	Frequency	Accuracy	Yes
A2SATRA [26]	Log linear analysis	No	Single	Frequency	Accuracy	Yes
2HESRA	Correlation analysis	5-fold	Multiple	Accuracy	Accuracy	Yes

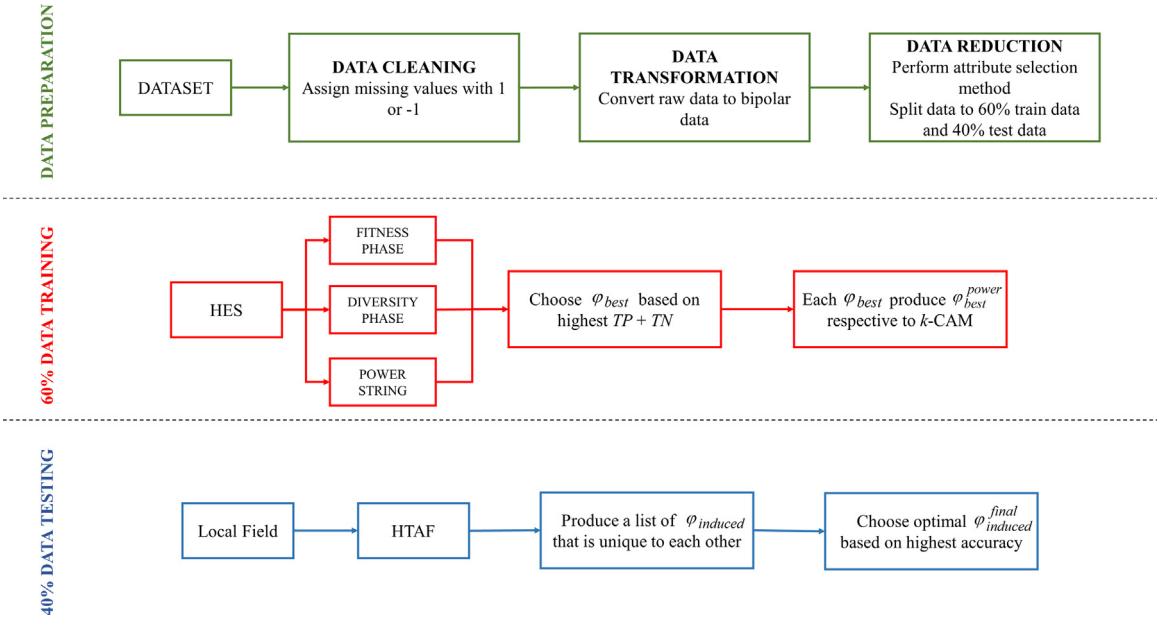

Fig. 1. Overall phases of 2HESRA in DHNN model.

Data Preparation Phase

1. Convert a raw dataset to a bipolar dataset $(1;-1)$ by using the k-means clustering method in SPSS software. Here, any missing values are randomly placed with 1 or –1.
2. Use correlation analysis to select the significant attributes of the dataset. Correlation method used Pearson's correlation coefficient to measure the strong association between outcome of a dataset and attributes [25,30]. Eq. (18) shows how $'_{MAJ2SAT}$ is formulated with optimal attributes S_i.

$$'_{MAJ2SAT} = \bigwedge_{i=1}^{p} S_i^{min\eth p_a\eth} \vee S_i^{min\eth p_b\eth} \bigwedge_{i=1}^{q} \times S_i^{min\eth p_c\eth} \vee S_i^{min\eth p_d\eth} \vee S_i^{min\eth p_e\eth} \quad (18)$$

   Noted that $S_i^{min\eth p_a\eth}$ is the optimal attribute being chosen since it achieves the lowest significant value from the correlation test. The attributes are selected based on $0 f p_a f "$ where " is the significant value that set to 0.5.
3. Split the dataset into a training dataset (60%) and a testing dataset (40%). Then, segregate the data using 5-fold cross-validation.

Data Training Phase

4. List all the possible combinations of the $'_{MAJ2SAT}$ and calculate the confusion matric for all training datasets.
5. Generate the top five $'_{best}$ based on the highest $TP + TN$. Each respective $'_{best}$ will produce five power best logic $'^{power}_{best}$ which are optimized by HES algorithm.
6. Store all $'^{power}_{best}$ in the CAM with optimal synaptic weight management.

Data Testing Phase

7. List all possible $'_{induced}$ based on the number of trials for each $'^{power}_{best}$.
8. After calculating the final neuron states, select the top five $'_{induced}$ that are unique and not redundant to others.
9. Calculate the accuracy based on the 40% testing datasets.
10. Select $'^{final}_{induced}$ based on highest percentage of accuracy.
11. The performance metrics will be calculated based on $'^{final}_{induced}$. The competency of the 2HESRA model will be compared with other models.

Fig. 1 depicts the overall implementation of proposed 2HESRA in DHNN model. The entire process includes three steps data preparation phase, data training phase, and data testing phase. Each phase has its own objective. In data preparation phase, we focus on finding significant attributes and data splitting. In data training phase, we generate $'^{power}_{best}$ for all CAM to ensure that the proposed model successfully achieve multi-objective functions before proceeding to the next phase. Lastly, the purpose of data testing phase is to generate $'^{final}_{induced}$ with optimal accuracy that best explains the information and decision of real datasets.

5. Experimental setup

This section explains the components of the experiments used to evaluate the optimization capabilities of 2HESRA in producing $'^{final}_{induced}$. All the algorithms are coded on C++ and run on an Intel® Core™ i7–1065G7 with 8 GB RAM. Notably, all algorithms are executed under the same system configuration.

Table 2
Details of benchmark models.

Code	Dataset	Total instances	Total attributes	Field of study	Goal
D1	Absenteeism at Work [31]	740	18	Business	1: Absent g 6:92 h −1: Otherwise
D2	Australian Credit Approval [32]	690	15	Financial	1: Good −1: Bad
D3	Autism Screening Adult [33]	704	21	Social	1: Autism −1: Not
D4	Automobile [34]	205	26	Automotive	1: Price g 13 207:13 −1: Otherwise
D5	Bach Choral Harmony [35]	5665	17	Music	1: Chord label g 3.13 −1: Otherwise
D6	Cylinder Bands [36]	541	40	Physical	1: Band −1: No band
D7	Bank Marketing [37]	4119	21	Business	1: Subscribed −1: Not
D8	Campus Recruitment [38]	215	14	Education	1: Placed −1: Not
D9	Chemical Composition of Ceramic Samples [39]	88	18	Physical	1: Body −1: Glaze
D10	Dermatology [40]	366	34	Life	1: Age g 36.3 −1: Otherwise
D11	Divorce Predictor [41]	170	55	Life	1: Divorced −1: Not
D12	Dry Bean [42]	13 611	17	Computer	1: Seker, Horoz & Sira −1: Barbunya, Bomabay, Cali & Dermason
D13	Engagement and job fit of retail salespeople [43]	2095	64	Business	1: Point of sales manager −1: Specialist
D14	Facebook metrics [44]	500	19	Business	1: Total interactions g 212 −1: Otherwise
D15	Statlog (Heart) [45]	270	14	Medical	1: Absence −1: Prescence
D16	Image Segmentation [46]	2310	19	Computer	1: Pixel classification g 0:13376 −1: Otherwise
D17	Ionosphere [47]	351	35	Physical	1: Good −1: Bad

5.1. Benchmark datasets

In this experiment, we have adopted 17 benchmark datasets (as shown in Table 2) to test different PAM models. With a good combination of different attributes (categorical, integer, real), these real-life datasets are very meaningful to managers, doctors, finance manager and others. The details of all datasets can be found from the UCI repository except for D8 and D13 which obtained from Kaggle. The motivation of choosing these 17 datasets is based on three key characteristics. First, it is important to use diverse datasets which comes from variety of subject area to have better insights. Second, the datasets should contain more than 80 instances to have a high-quality train–test split data. For example, if a dataset consists of 30 train data, this will be resulting to poor performance. Third, the datasets should have more than 10 attributes in order to have optimal attribute selection.

Table 3
Details of categories in confusion matrix.

Classification	Definition	Predicted	Actual outcome
TP (True Positive)	The number of instances correctly classified as positive by the model.	Positive	Positive
TN (True Negative)	The number of instances correctly classified as negative by the model.	Negative	Negative
FP (False Positive)	The number of instances incorrectly classified as positive by the model.	Positive	Negative
FN (False Negative)	The number of instances incorrectly classified as negative by the model.	Negative	Positive

Additionally, the continuous values in a dataset are handle by assigning them with 1 or –1 according to the average values calculated by k-means clustering in SPSS software. Meanwhile, the missing values are treated by using data imputation method. The missing values will be randomly replaced with either 1 or –1 since it is impractical to remove the entry from a dataset.

5.2. Performance metrics

In our experiments, we use several metrics to construct the performance evaluation system, including accuracy (*ACC*), sensitivity (*SEN*), precision (*PRE*), and Matthew's correlation coefficient (*MCC*). Table 3 shows the important values used to calculate performance metrics in evaluating the effectiveness of the proposed model.

The *ACC* metric is considered a widely used performance metric and is calculated using Eq. (19). It is calculated by dividing the number of correct predictions (true predictions) by the total number of samples predicted by the model [48].

$$\mathrm{ACC} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}} \tag{19}$$

In classification problems, *SEN* and *PRE* are significant metrics for evaluating performance of a model. *SEN* quantifies the value of actual positive instances correctly identified by a classifier. In perspective of D2 dataset [49], by adopting a classifier with higher *SEN*, financial institutions can potentially minimize their financial losses. Meanwhile, *PRE* quantifies the accuracy of positive predictions made by a model [50]. A higher *PRE* value indicates fewer *FP* predictions. Both *SEN* and *PRE* can be formulated as in Eqs. (20)–(21), respectively:

$$\mathrm{SEN} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \tag{20}$$

$$\mathrm{PRE} = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}} \tag{21}$$

Furthermore, *MCC* considers all *TP*, *TN*, *FP*, *FN* predictions to provide a comprehensive measure of a classifier performances as formulated in Eq. (22). It gives a high score when the classifier achieves accuracy in predicting positive and negative instances, as well as ensuring that its positive and negative predictions are mostly correct. Notably, *MCC* is a valuable metric as it considers the overall performance of the classifier across all prediction outcomes [51].

$$\mathrm{MCC} = \frac{\mathrm{TP}\,\mathrm{TN} - \mathrm{FP}\,\mathrm{FN}}{\sqrt{(\mathrm{TP} + \mathrm{FP})(\mathrm{TP} + \mathrm{FN})(\mathrm{TN} + \mathrm{FP})(\mathrm{TN} + \mathrm{FN})}} \tag{22}$$

5.3. Baseline models

The primary focus of this paper is to assess the performance of the induced logic generated by 2HESRA. Therefore, our comparison is specifically limited to methods that also produce induced logic. As a result, we will not be comparing 2HESRA with other types of models such as decision trees, linear regression, or random forest since they do not incorporate SAT logic. Instead, our comparison will be centred around three recently developed RA models: A2SATRA, P2SATRA and S2SATRA. In this section, we will delve into the implementation details of each of these models, providing a comprehensive analysis for the purpose of comparison.

(a) Jamaludin et al. [24] proposed P2SATRA which focused on implementing logical permutation operator in the model. The objective of proposing permutation operator is to increase the solution space thus have high chance of generating high *ACC*. This model will permutate the attribute arrangement by ensuring it will not be redundant to each other. Other than that, the mechanism of P2SATRA is similar to conventional 2SATRA [22,23]. The results show that P2SATRA outperform 2SATRA.
(b) Ref. [25] proposed S2SATRA by improving the structure of P2SATRA. The data preparation phase is different with 2SATRA and P2SATRA since this is the first effort to implement correlation analysis to choose significant attributes. By investigating the association between outcome and attributes, strongest association attributes are chosen. From the study, S2SATRA successfully outperform other models.
(c) Ref. [26] proposed A2SATRA which inspired by S2SATRA that implements log linear analysis as attribute selection method in data preparation phase. The model also implements permutation operator same as P2SATRA and S2SATRA. The finding shows that A2SATRA is competitive as S2SATRA.

5.4. Parameters assignments

In this section, we will present the parameters assignments used for all models. In some cases, there are different configuration used for each model to maintain the originality of the model. Tables 4 until 7 depicts the different parameter assignment used for 2HESRA, A2SATRA, P2SATRA, and S2SATRA. It is important to note that 2HESRA is the only PAM model that utilize multi-objective optimization technique. Therefore, the search space of 2HESRA has increased from single CAM to multiple CAM. The number of attributes for 2HESRA is different because of the presence 2SAT and 3SAT in $'_{MAJ2SAT}$. The number of attributes cannot be similar to other models since the lowest combination of $'_{MAJ2SAT}$ is seven. The datasets will be split to 60% training data and 40% testing data. Each data will be performed 5-fold cross validation to ensure that data is trained, tested and evaluated five times to avoid any potential bias. In data training phase, the output is the production of $'^{power}_{best}$. 2HESRA will produce five $'_{best}$ for each logical combination. The $'_{best}$ logic with highest $TP + TN$ will be chosen as $'^{power}_{best}$. In other models, $'^{power}_{best}$ is not exist due to single CAM. The stated models will choose $'_{best}$ based on the highest frequency of 2SAT logic. During data testing, by using $'^{power}_{best}$ and $'_{best}$, the simulation will undergo 100 trials to produce list of $'_{induced}$. The $'^{final}_{induced}$ is chosen based on highest *ACC*. 2HESRA will utilize Eq. (18) to choose $'^{final}_{induced}$. Meanwhile, other models choose $'^{final}_{induced}$ based on the highest success of $'_{induced} = '_{test}$. Besides that, the limitation of A2SATRA, P2SATRA and S2SATRA is they only consider $'_{train} = 1$ during data training phase.

Table 4
The parameter assignments used for 2HESRA model.

Parameter	Value
Number of CAM	5
Diversity threshold	0.1
Number of attributes	7
Training algorithm	HES
Attribute selection	Correlation
Logical combination	100
Logical permutation	100
Population size	100
Training iteration	100
Trial	100
k-fold cross-validation	5
Train:test split ratio	60:40

Table 5
The parameter assignments used for A2SATRA model [26].

Parameter	Value
Number of CAM	1
Diversity threshold	–
Number of attributes	6
Training algorithm	ES
Attribute selection	Log–linear
Logical combination	100
Logical permutation	100
Population size	100
Training iteration	100
Trial	100
k-fold cross-validation	5
Train:test split ratio	60:40

Table 6
The parameter assignments used for P2SATRA model [24].

Parameter	Value
Number of CAM	1
Diversity threshold	–
Number of attributes	6
Training algorithm	ES
Attribute selection	Random
Logical combination	100
Logical permutation	100
Population size	100
Training iteration	100
Trial	100
k-fold cross-validation	5
Train:test split ratio	60:40

Table 7
The parameter assignments used for S2SATRA model [25].

Parameter	Value
Number of CAM	1
Diversity threshold	–
Number of attributes	6
Training algorithm	ES
Attribute selection	Correlation
Logical combination	100
Logical permutation	100
Population size	100
Training iteration	100
Trial	100
k-fold cross-validation	5
Train:test split ratio	60:40

6. Results and discussion

After the full implementation of 2HESRA in DHNN model, we will compare its performance with three existing models which are A2SATRA, P2SATRA, and S2SATRA based on *ACC*, *SEN*, *PRE*, and *MCC* metrics. The detail descriptions for all performance metrics will be analysed and discussed based on Tables 8 until 11 and Figs. 2 until 5.

Table 8
The values of *ACC* by all logic mining models.

Datasets	2HESRA	A2SATRA	P2SATRA	S2SATRA
D1	**0.6743**	0.6371	0.5703	0.5761
	0.0000	0.0584	0.1824	0.1705
D2	**0.8152**	0.6600	0.6333	0.6333
	0.0000	0.2352	0.2872	0.2872
D3	0.7099	0.7081	0.7045	**0.8574**
	−0.2078	0.2108	0.2170	0.0000
D4	**0.8878**	0.6976	0.6976	0.6683
	0.0000	0.2726	0.2726	0.3284
D5	**0.6483**	0.5984	0.5956	0.6135
	0.0000	0.0834	0.0885	0.0567
D6	**0.7157**	0.6153	0.6654	0.6139
	0.0000	0.1632	0.0756	0.1658
D7	**0.8246**	0.7191	0.7826	0.7922
	0.0000	0.1467	0.0537	0.0409
D8	**0.7628**	0.7052	0.7052	0.6977
	0.0000	0.0817	0.0817	0.0933
D9	0.9200	0.8667	**1.0000**	0.8114
	−0.0870	0.1538	0.0000	0.2324
D10	**0.6164**	0.5814	0.5841	0.5863
	0.0000	0.0602	0.0553	0.0513
D11	**0.9824**	0.9618	0.9706	0.9706
	0.0000	0.0214	0.0122	0.0122
D12	**0.8161**	0.5887	0.6573	0.6992
	0.0000	0.3863	0.2416	0.1672
D13	**0.5928**	0.5706	0.5616	0.5702
	0.0000	0.0389	0.0556	0.0396
D14	**0.8120**	0.6827	0.6457	0.7850
	0.0000	0.1894	0.2575	0.0344
D15	**0.7741**	0.7407	0.7481	0.6981
	0.0000	0.0451	0.0348	0.1089
D16	**0.9167**	0.7071	0.8286	0.8405
	0.0000	0.2964	0.1063	0.0907
D17	**0.8357**	0.6157	0.8043	0.8007
	0.0000	0.3573	0.0390	0.0437
Average	**0.7826**	0.6857	0.7150	0.7185
Minimum	**0.5928**	0.5706	0.5616	0.5702
Maximum	0.9824	0.9618	**1.0000**	0.9706
W/E/L	15/0/2	0/0/17	1/0/16	1/0/16

As seen by the above-mentioned results in Table 8, 2HESRA model has an *ACC* as high as 98.24%, with an average 78.26% for all datasets. 2HESRA achieved the highest *ACC* in 15 out of 17 datasets. The second highest model that competes with 2HESRA is S2SATRA which gives an average of 71.85%. Also, P2SATRA has a slightly lower average value compared to S2SATRA with an average of 71.5%. According to Fig. 2, no value for *ACC* less than 59% is recorded. This justifies that 2HESRA capabilities in differentiating *TP* and *TN* value. The Friedman test is conducted for all datasets with a significance level of 0.05 and a degree of freedom equal to three. The *p*–value for *ACC* is $p = 18{:}3706$, with a chi-square value of $^2 = 26{:}4353$. Consequently, the null hypothesis of equal performance for all models is rejected. It is worth mentioning that HES algorithm with intelligent mutation helps 2HESRA to correctly predict *TP* and *TN*. This is because this operator only mutated the unsatisfied clauses which helps in producing optimal $'_{best}$. Other than that, 2HESRA has an effective technique in producing $'^{power}_{best}$ since the model will select top five $'_{best}$ based with different logical combination. These $'_{best}$ will be stored in respective CAM. In each $'_{best}$, five more top logic will be learned and produced in the training phase based on highest $TP + TN$. This technique is clearly different with other existing models that choose $'_{best}$ based on the frequency of 2SAT clauses.

The results presented in Fig. 3 demonstrate that the 2HESRA model exhibits an impressive *SEN* of 100%, with an overall average of 71.81% across all datasets. Out of the 17 datasets, 2HESRA achieved the highest *SEN* in 13 of them, making it the top-performing model. Coming in second is the S2SATRA model, which boasts an average *SEN* of 59.67%. Additionally, the P2SATRA model has a slightly lower average *SEN* of 57.98% compared to S2SATRA. In Table 9, D16 dataset achieves *SEN* value equal to 1 which means 2HESRA can correctly predict positive result during the testing phase of DHNN. The Friedman test was performed on all datasets, using a significance level of 0.05 and degrees of freedom equal to three. The *p*–value for *SEN* was found to be $p = 0{:}00037$, while the chi-square value was $^2 = 18{:}3706$. As a result, the

Fig. 2. The spider-web graph of *ACC* values attained by all models.

Fig. 3. The spider-web graph of *SEN* values attained by all models.